

A Simulation-Based Evaluation of Cache Eviction Policies for Distributed Inference Systems

Andrea Quiroz
Distributed Computing Lab
EPFL
Lausanne, Switzerland
laura.quirozortega@epfl.ch

Gauthier Voron
Distributed Computing Lab
EPFL
Lausanne, Switzerland
gauthier.voron@epfl.ch

Rachid Guerraoui
Distributed Computing Lab
EPFL
Lausanne, Switzerland
rachid.guerraoui@epfl.ch

Abstract—Distributed inference systems for machine learning models face increasing memory pressure due to model size, hierarchical memory constraints, and emerging multi-model workloads. In this work, we study how classical cache eviction policies impact inference latency in distributed serving environments. We design and implement an event-driven simulator that models distributed model execution, hierarchical memory systems, and configurable cache replacement strategies, and we evaluate six policies across multiple workload distributions and serving scenarios. Our results show that no single policy is universally optimal, with LFU and MRU consistently achieving the best performance under different conditions, while workload skew strongly determines both median and tail latency behavior.

Index Terms—distributed inference, cache memory, replacement policy, performance evaluation.

I. INTRODUCTION

Large Language Models (LLMs) and other foundation models have become a critical component of modern AI systems. They are used across a wide range of applications, including conversational assistants, Retrieval-Augmented Generation (RAG), autonomous agents, code generation, and multimodal reasoning systems [1, 2, 3]. Serving these models requires executing inference requests with low latency while handling large numbers of concurrent users. As AI adoption continues to grow, inference serving has become one of the primary operational challenges in modern computing infrastructure.

A major challenge of model serving is the size of contemporary AI models. State-of-the-art models contain billions of parameters and require substantial memory resources to store both model weights and runtime state such as Key-Value (KV) caches [4, 5, 6]. For example, a single large model may require hundreds of gigabytes of memory [7], often exceeding the capacity of a single machine. Furthermore, inference workloads exhibit temporal locality: models that are requested once are likely to be requested again in the near future, while popular models may dominate large fractions of the request stream [1, 8]. Efficient management of model residency therefore becomes essential for achieving low-latency inference.

To address these resource constraints, modern serving systems increasingly rely on distributed inference architectures, where model execution is partitioned across multiple machines and memory hierarchies [4, 5]. In such environments, model

layers must be transferred between disk, main memory, and GPU memory before inference can proceed. Since memory resources are limited, workers cannot keep every model permanently resident. Consequently, the system must continuously decide which models or layers should remain in memory and which should be evicted to make room for incoming requests.

This challenge is becoming increasingly important as AI systems evolve from single-model deployments to compound AI systems [2]. Traditional LLM serving typically requires a single model per request, whereas modern RAG pipelines, agentic workflows, and multimodal applications often invoke multiple models within a single inference pipeline [9, 1, 3]. These workloads increase memory pressure, amplify model switching costs, and make cache management decisions significantly more impactful on end-to-end latency.

Existing research on inference serving has primarily focused on scheduling [4, 6], memory offloading [5], workload characterization [8], and resource provisioning [10]. In contrast, comparatively less attention has been devoted to understanding how classical cache eviction policies behave when applied to distributed model serving workloads. While replacement policies such as Least Recently Used (LRU), Least Frequently Used (LFU), First-In-First-Out (FIFO), and Random Replacement (RR) are widely used in computer architecture and content distribution networks [11, 12], their effectiveness for distributed inference workloads remains poorly understood.

In this work, we investigate the impact of cache eviction policies on distributed inference latency. We design and implement an event-driven simulator that models clients, a centralized coordinator, distributed workers, hierarchical memory systems, and configurable cache eviction strategies. Using this simulator, we evaluate multiple cache eviction policies under a variety of workload distributions and serving patterns representative of modern AI deployments.

Our contributions are summarized as follows:

- We formalize cache management in resource-constrained distributed inference systems, modeling end-to-end inference latency as a function of network transfer, memory hierarchy movement, computation, and eviction decisions.
- We design and implement a concurrent event-driven simulator in Go that reproduces distributed inference execu-

tion, including hierarchical memory (disk/RAM/vRAM), concurrent request handling, and fully pluggable cache eviction policies [13].

- We evaluate six classical cache eviction policies (LRU, LFU, FIFO, LIFO, MRU, and RR) under five workload distributions and two serving scenarios (single-model and multi-model inference), resulting in a comprehensive evaluation over 10 independent runs per configuration.
- We show that cache retention is critical for single-model serving workloads. Compared to immediate eviction, caching reduces median inference latency by more than three orders of magnitude (71,486.6s vs 34–36s). Among the evaluated policies, LFU achieves the lowest median latency, while MRU and LIFO provide the best tail-latency performance.
- We demonstrate that multi-model serving amplifies memory contention and increases the importance of eviction decisions. In this setting, LFU and MRU achieve the lowest median latency (0.521s), while MRU provides the best overall tail-latency performance.
- We show that workload characteristics strongly influence cache effectiveness. While no policy is optimal for all workloads, LFU consistently achieves the most robust performance across both serving scenarios and workload distributions, making it the strongest general-purpose policy among those evaluated.

The remainder of this paper is organized as follows. Section II presents the background on inference serving, workload characteristics, and cache replacement policies. Section III describes the system design and problem formulation. Section IV details the simulator implementation. Section V presents the experimental evaluation and answers the research questions. Finally, Section VI concludes the paper and discusses future work.

II. BACKGROUND

A. Distributed Inference and Model Serving

Inference is the process of executing a trained machine learning model to generate predictions from an input request. Modern large language models (LLMs) perform inference autoregressively, generating output tokens one at a time while maintaining a Key-Value (KV) cache that stores intermediate attention states from previously generated tokens [4, 6]. This cache avoids recomputing the entire sequence during generation and is therefore essential for efficient inference serving.

Serving modern foundation models requires significant computational and memory resources. Model weights can occupy hundreds of gigabytes, while the KV cache grows with the length of the conversation and generated output [4, 5]. Since these requirements often exceed the capacity of a single hardware unit, inference systems distribute execution across multiple machines and memory tiers. Such systems must continuously manage model placement and movement across vRAM, RAM, and persistent storage to ensure that the required model data is available when requests arrive [5].

Traditionally, inference serving focused on a single model that handled an entire request. However, modern AI applications increasingly rely on compound systems involving multiple models and services. Retrieval-Augmented Generation (RAG), multimodal systems, and agentic workflows frequently require a single user request to invoke several models, retrievers, or auxiliary services before producing a response [1, 2]. Consequently, inference infrastructure must support both single-model and multi-model serving patterns while minimizing the latency introduced by model loading and memory management.

B. Model Serving Patterns

Model serving workloads can be broadly categorized into single-model and multi-model serving scenarios.

a) Single-Model Serving.: In single-model serving, each request is processed by a single model instance. This pattern is representative of traditional LLM applications such as conversational assistants, text generation systems, and classification services. Performance primarily depends on efficient scheduling, memory management, and reuse of previously loaded model data [4, 6].

b) Multi-Model Serving.: In multi-model serving, a request may require the execution of multiple models within a processing pipeline. Examples include RAG systems that combine embedding models with generation models, multimodal systems that process images and text jointly, and agentic frameworks that orchestrate multiple specialized components [1, 3, 2]. Compared to single-model serving, these workloads place greater pressure on memory resources because multiple models may need to be resident simultaneously or loaded repeatedly throughout the execution of a request.

The distinction between these serving patterns is particularly important for cache management. Single-model workloads tend to exhibit stronger reuse of a small set of models, whereas multi-model workloads increase memory contention and introduce more complex residency decisions.

C. Workload Characteristics

The effectiveness of a cache eviction policy depends heavily on the characteristics of the incoming workload. Prior studies have shown that AI serving workloads often exhibit temporal locality, where recently accessed data is likely to be accessed again in the near future [1]. Additionally, request popularity is frequently skewed, with a small subset of models or documents receiving a disproportionate fraction of requests [12, 1].

To capture a range of realistic serving conditions, workload generation is commonly modeled using statistical distributions. Uniform distributions represent workloads without popularity bias, while Normal distributions model moderate concentration around a subset of frequently requested models. Power Law distributions capture highly skewed popularity patterns and are widely used to model real-world request behavior. Poisson processes are commonly used to model independent request arrivals, whereas Gamma distributions are often employed

to represent bursty traffic patterns observed in production environments [8, 4, 10].

Because cache performance is closely tied to access locality and request skew, evaluating cache policies across multiple workload distributions is necessary to understand their behavior under different operating conditions.

D. Cache Replacement Policies

When the capacity of a cache is exhausted and new data must be inserted, the system must decide which existing data should be removed. This decision is governed by a cache replacement policy.

The theoretically optimal replacement policy, commonly referred to as OPT, evicts the item whose next access occurs farthest in the future. Although OPT minimizes cache misses, it requires perfect knowledge of future requests and is therefore impossible to implement in practice [11]. As a result, real systems rely on heuristic policies that approximate optimal decisions using information about past accesses or residency patterns.

Cache replacement policies have been widely studied across multiple domains, including processor caches, operating systems, storage systems, content delivery networks, and distributed computing platforms [11, 12]. Common policies include Random Replacement (RR), Least Recently Used (LRU), Least Frequently Used (LFU), First-In-First-Out (FIFO), Last-In-First-Out (LIFO), and Most Recently Used (MRU). Each policy makes different assumptions regarding future access behavior, resulting in varying performance depending on the workload characteristics and available memory resources.

In distributed inference systems, cache replacement policies play a critical role in determining which model layers remain resident in memory and which layers must be reloaded from slower storage tiers. Consequently, the choice of replacement policy directly influences model availability, memory utilization, and inference latency.

III. DESIGN

A. Problem Motivation

Distributed inference systems accelerate AI model serving by partitioning inference execution across multiple interconnected worker nodes. In such systems, models are distributed across heterogeneous memory hierarchies composed of vRAM, RAM, and disk. To execute inference requests efficiently, the required model layers must be available in the appropriate memory tier at the time of execution.

Modern foundation models introduce significant memory pressure due to their large weight sizes and KV cache requirements. Reloading model data from persistent storage or transferring it across the network for every request introduces substantial latency overhead. At the same time, inference workloads often exhibit temporal locality: models that have been requested recently are likely to be requested again in the near future. This creates an opportunity for cache-aware

memory management strategies to reduce repeated transfer costs and improve request latency.

The objective of this work is to identify cache eviction policies that minimize inference latency in resource-constrained distributed inference environments.

B. Problem Formulation

We model the distributed inference system as a set of worker nodes

$$W = \{w_1, w_2, \dots, w_n\},$$

where each worker maintains a hierarchical memory system composed of three memory tiers:

$$tier \in \{vRAM, RAM, Disk\}.$$

For each worker $w_j \in W$, let the memory capacity vector be defined as

$$D_j = (D_j^{vRAM}, D_j^{RAM}, D_j^{Disk}),$$

representing the storage capacity of each memory tier. Let

$$M = \{m_1, m_2, \dots, m_k\}$$

denote the set of available models. Each model m_i is composed of a sequence of layers

$$m_i = \{l_1^{(i)}, l_2^{(i)}, \dots, l_{p_i}^{(i)}\},$$

where each layer contains model weights and KV cache data.

Let

$$\mathcal{L}_j(t)$$

denote the set of layers currently assigned to worker w_j at time t .

Let

$$R(l, w_j, tier, t) \in \{0, 1\}$$

denote the residency state of layer l on worker w_j within memory tier $tier$ at time t , where $R = 1$ indicates that the layer is currently stored in the corresponding memory tier.

The memory utilization of worker w_j within a memory tier is defined as

$$U_j^{tier}(t) = \sum_{l \in \mathcal{L}_j(t)} R(l, w_j, tier, t) \cdot size(l),$$

where $size(l)$ denotes the memory footprint of layer l composed as the sum of the weight size and the KV cache size.

Inference requests arrive over time according to a workload distribution $\lambda(t)$. Each request may require the execution of one or more models, depending on the serving scenario. Model

execution proceeds sequentially across the layers associated with each requested model.

Let $L(r)$ denote the latency of inference request r , defined as the elapsed time between the client issuing the request and receiving the final response.

The latency of a request can be decomposed as

$$L(r) = L_{\text{network}}(r) + L_{\text{disk}}(r) + L_{\text{memory}}(r) + L_{\text{compute}}(r) + L_{\text{eviction}}(r)$$

where the individual terms correspond to network transfer cost, disk loading cost, memory transfer cost, inference execution cost, and cache eviction overhead, respectively.

The objective of the cache policy is to minimize the latency experienced by inference requests under the workload distribution $\lambda(t)$:

$$\min_{\theta} \mathbb{E}_{r \sim \lambda(t)}[L(r)],$$

subject to the memory capacity constraints of each worker:

$$U_j^{\text{tier}}(t) \leq D_j^{\text{tier}},$$

for all workers $w_j \in W$.

The optimization variable θ defines the cache eviction policy responsible for managing layer residency across the hierarchical memory system. Given memory pressure, θ determines which layers are retained, promoted, demoted, or evicted from each memory tier over time.

C. Design Rationale

The objective of this work is to understand how cache eviction policies influence inference latency in distributed AI serving systems. Because cache performance depends not only on the eviction strategy itself but also on workload characteristics and execution patterns, a controlled experimental framework is required to isolate the effect of each factor.

To achieve this, the evaluation varies two dimensions that directly influence cache behavior: serving configuration and workload distribution. The serving configuration determines how models are used during inference. We consider both single-model serving, representative of traditional inference workloads, and multi-model serving, representative of emerging applications such as retrieval-augmented generation and agentic systems, where multiple models may participate in processing a single request. These two scenarios exhibit different levels of model reuse and memory contention, which can significantly affect cache effectiveness.

The workload distribution determines how requests are generated over time. We evaluate five distributions—Uniform, Normal, Power Law, Poisson, and Gamma—to capture a broad spectrum of request patterns commonly observed in AI serving systems. Together, these distributions model different levels of popularity skew, locality, and traffic burstiness, ranging from uniformly accessed workloads to highly concentrated access patterns and bursty request arrivals.

By systematically varying these two dimensions while keeping the execution model unchanged, performance differences can be attributed directly to the cache policy, the workload characteristics, or the serving configuration. This enables a fair comparison of eviction strategies under diverse but controlled operating conditions.

Due to the complexity of distributed inference systems and the strong interaction between memory management, data movement, and execution, analytical evaluation of cache policies is impractical. Consequently, we adopt a simulation-based methodology that provides consistent execution semantics across all experiments while enabling large-scale and reproducible evaluation.

D. Cache Policy Abstraction

To support fair comparison between policies, cache eviction strategies are implemented through a shared policy interface exposed by the Memory Manager (further discussed in IV-B6). The interface abstracts insertion, lookup, and eviction operations while allowing policies to maintain policy-specific metadata such as access timestamps, insertion timestamps, or frequency counters.

This design allows eviction policies to be exchanged without modifying the remaining simulator components, ensuring that all policies are evaluated under identical execution and memory management conditions.

IV. IMPLEMENTATION

A. Simulator Overview

To evaluate cache eviction policies under distributed inference workloads, we implemented an event-driven simulator in Go [13]. The simulator models a distributed inference system composed of clients, a centralized coordinator, and a pool of workers with hierarchical memory capacities. The implementation was designed to support configurable cache policies, concurrent request orchestration, and detailed event logging for post-execution analysis.

Go was chosen due to its lightweight concurrency primitives and efficient support for large numbers of concurrent tasks through goroutines and channels. While the simulator orchestrates requests concurrently, execution costs are tracked using a global logical clock to provide a deterministic and reproducible view of system behavior. The simulator prioritizes analytical reproducibility and policy comparison over cycle-accurate hardware modeling.

B. System Architecture

1) *Inference Execution Flow*: An inference request begins when a client submits a model request to the coordinator. The coordinator determines whether the required model layers are already available within the worker cluster. If some layers are missing, the coordinator initiates a loading phase in which the corresponding workers fetch the required layers into their local memory hierarchies. If the model is not present in the system, the client first uploads it to the coordinator, which partitions the model and distributes its layers across workers.

Once all required layers are available, inference execution proceeds sequentially through the model. Each worker executes the subset of layers assigned to it and returns a completion signal to the coordinator, which triggers the next stage of execution. After all layers have been processed, the final inference result is returned to the client.

2) *Client*: Clients represent the workload generation layer of the simulator. Each client issues inference requests to the coordinator, acting as the entry point for all simulated execution.

In the implementation, clients are executed as concurrent goroutines that generate requests according to the configured workload pattern. Each request is sent to the coordinator through a communication channel, where it enters the system’s scheduling pipeline.

3) *Coordinator*: The coordinator is responsible for system-wide orchestration. It maintains a global registry of model-layer locations, distributes layers across workers using a deterministic mapping strategy, and coordinates model loading and inference execution.

Upon receiving a request, the coordinator consults its registry to determine layer availability and issues the corresponding commands to workers. During execution, it synchronizes the sequential progression of inference across workers and collects completion signals until the request has been fully processed.

4) *Worker*: Each worker represents a compute node with dedicated storage and a local memory hierarchy. Workers are responsible for executing inference operations assigned by the coordinator.

During execution, each worker continuously listens for incoming instructions from the coordinator. Upon receiving a request, it delegates memory management operations to its local Memory Manager to ensure that the required model layers are loaded into the appropriate memory tier before computation.

Once the necessary data is available in vRAM, the worker simulates the execution of the corresponding model layers. After completing its assigned computation, it sends a completion signal back to the coordinator to continue the inference pipeline.

5) *Model*: A model is represented as a sequence of discrete layers. Each layer consists of two components: the weight parameters and the associated KV cache. The size of a layer is defined as the sum of the memory footprint of its weights and its KV cache, which directly impacts its placement within the memory hierarchy.

Each model and its layers are assigned unique identifiers to enable tracking and mapping across worker nodes. This allows layers to be independently managed and distributed across the system during execution.

6) *Memory Manager*: Each worker is associated with a local Memory Manager responsible for tracking model residency across a hierarchical memory system and enforcing capacity constraints. The memory system is organized into three tiers: disk, RAM, and vRAM.

Data movement between tiers follows a hierarchical transfer process. Data must first be loaded from disk into RAM before it can be promoted to vRAM. Since inference execution is performed exclusively in vRAM, the Memory Manager ensures that all required layers are present in this tier before computation can proceed.

When a worker requests a model layer, the Memory Manager checks its internal registry to determine the current location of the layer. If the layer is not yet present in the system, it is loaded into disk and subsequently promoted to vRAM following the defined hierarchy.

To model realistic contention in shared GPU environments, layers residing in vRAM are subject to a locking mechanism during active inference. A layer marked as ‘Locked’ cannot be selected for eviction, ensuring that ongoing computations are not interrupted. Once inference completes, the layer is unlocked and becomes eligible for eviction.

When a memory tier reaches capacity, the Memory Manager invokes an eviction policy to select a victim from the set of unlocked layers. The memory subsystem exposes a policy-agnostic interface for insertion, lookup, and eviction operations. This design allows different eviction strategies to be swapped without modifying the core simulator logic.

7) *System*: All nodes in the system (clients, coordinator, and workers) share a common system abstraction responsible for global time management and event logging. The system serves two main purposes: maintaining a logical clock and recording all operations for post-execution analysis.

a) *Logical Clock*: The simulator uses a deterministic, additive virtual clock to model time progression. In this approach, every operation in the system increments a single global counter based on its simulated cost. This creates a strictly sequential timeline in which all events are ordered consistently.

While the implementation uses Go goroutines to orchestrate concurrent execution, the simulated timeline abstracts away physical parallelism. Instead, the system models execution as a single logical pipeline, prioritizing reproducibility and analytical clarity over hardware-level interleaving.

b) *Event Logging and Cost Model*: A centralized logger records all system operations and computes their associated execution costs. These costs determine how the logical clock is advanced and provide the basis for performance analysis.

The system distinguishes four types of operations: network transfer, disk transfer, memory transfer, and inference. Each operation type has an associated cost model:

- Network transfer: computed as $\frac{\text{data size}}{\text{network bandwidth}} + \text{latency}$.
- Disk transfer: computed as $\frac{\text{data size}}{\text{disk bandwidth}}$.
- Memory transfer: computed as $\frac{\text{data size}}{\text{vRAM bandwidth}}$.
- Inference: defined as a predefined cost associated with each model request. This value is provided as part of the simulation input and is not computed during execution.

These costs are used both to advance the logical clock and to timestamp events in the execution log, enabling fine-grained post-hoc analysis of system behavior.

C. Concurrency Model

The simulator uses Go’s concurrency primitives to model asynchronous interactions between distributed components. Concurrency is achieved through goroutines, while communication between components is implemented using channels.

The coordinator handles incoming client requests concurrently by processing each request in a separate goroutine. Similarly, workers process incoming execution instructions concurrently, allowing multiple inference tasks to be simulated in parallel from an orchestration perspective.

Communication between system components is performed exclusively through channels. Clients send requests to the coordinator via channels, the coordinator dispatches execution instructions to workers through channels, and workers notify the coordinator upon completion of execution using return channels.

To ensure consistency of shared state, mutexes are used to protect critical sections of the system. A global mutex guards updates to the logical clock, while the coordinator uses a mutex to protect its model residency registry. Additionally, each worker applies mutex protection to its Memory Manager to prevent concurrent modifications of memory state during overlapping requests.

This design enables the simulator to model asynchronous distributed execution while maintaining deterministic and thread-safe updates to shared system state.

D. Cache Policy Implementation

The cache eviction policies are implemented as modular components independent of the core Memory Manager logic. Each policy conforms to a shared eviction interface and may be applied to any memory tier. Policy-specific metadata is maintained to support eviction decisions, including access timestamps, insertion timestamps, and frequency counters depending on the policy.

Eviction is triggered when a worker attempts to load data into a full vRAM tier. This may initiate a cascading eviction process across lower memory tiers to ensure sufficient space is available. To account for contention with active inference tasks, vRAM enforces a locking mechanism: layers currently participating in inference are marked as locked and cannot be selected as eviction candidates. This constraint does not apply to RAM or disk tiers. To handle transient contention, each eviction attempt includes a retry mechanism with up to five attempts and a fixed backoff delay, after which an Out-of-Memory (OOM) error is raised if no valid eviction candidate becomes available.

a) Immediate Eviction: Under the Immediate policy, model layers are not retained after inference execution. Once a layer has been loaded into vRAM and used to process a request, it is immediately removed from memory rather than remaining available for future requests. Consequently, subsequent requests for the same model require the layer to be resent through the network and reloaded through the memory hierarchy before inference can proceed. This policy effectively

disables caching and serves as a lower-bound baseline for evaluating the benefits of cache retention.

b) Random Replacement (RR): RR first filters all layers residing in the target storage tier that are eligible for eviction. In vRAM, this set excludes currently locked layers; in lower tiers, all layers are eligible. From the resulting candidate set, a victim is selected using the simulator’s pseudo-random number generator. This avoids any need for ordering metadata such as timestamps or frequency counters.

c) Least Recently Used (LRU): LRU maintains an access log that stores a timestamp for each layer access. The policy iterates over all eligible layers in the target tier and selects the layer with the oldest (minimum) access timestamp. The access log is updated at the moment a layer is requested for use. If a layer is evicted to a different memory tier, its access timestamp is preserved unchanged.

d) Least Frequently Used (LFU): LFU tracks access frequency. The policy selects the layer with the lowest access count among eligible candidates. To ensure deterministic behavior, ties are broken using the access log: the layer with the oldest access timestamp is selected. Frequency counters are not reset when a layer is moved between memory tiers.

e) First-In-First-Out (FIFO): FIFO tracks residency duration using an insertion log, which records the timestamp at which a layer entered a given memory tier. The policy evicts the eligible layer with the earliest insertion timestamp. The insertion timestamp is updated every time a layer transitions between memory tiers, ensuring that FIFO reflects residency within the current tier only.

f) Last-In-First-Out (LIFO): LIFO uses the same insertion log as FIFO but selects the eligible layer with the most recent insertion timestamp. This effectively reverses FIFO behavior while maintaining identical metadata requirements.

g) Most Recently Used (MRU): MRU relies on the access log and selects the eligible layer with the most recent (maximum) access timestamp, evicting the most recently accessed data among the candidate set.

V. EVALUATION

This section evaluates the behavior of the implemented cache eviction policies under different distributed inference workloads. The evaluation is guided by the following research questions: Which cache eviction policy minimizes inference latency for single-model serving workloads under constrained memory resources V-E1? Which cache eviction policy minimizes median inference latency for multi-model serving workloads under constrained memory resources V-E2? How does workload distribution affect cache eviction policy performance in terms of inference latency V-E3?

A. Baselines

We compare the evaluated cache eviction policies against one baseline configuration:

- Immediate Eviction. After a worker completes inference execution, all model layers associated with the request are immediately removed from memory. Subsequent requests

therefore require the model to be re-uploaded by the client, redistributed by the coordinator, and reloaded into the worker memory hierarchy.

B. Experimental Setup

This section summarizes the simulated hardware, network, and workload configurations used in the evaluation.

1) *Hardware Configuration*: The simulated hardware configuration is based on the NVIDIA DGX H100 standard configuration for inference-oriented workloads [14]. Memory transfer bandwidths were derived from PCIe Gen5 specifications [15].

TABLE I
SIMULATED HARDWARE CONFIGURATION

Parameter	Value
vRAM Capacity	640 GB
RAM Capacity	2 TB
Disk Capacity	34.56 TB
RAM ↔ vRAM Bandwidth	128 GB/s
Disk ↔ RAM Bandwidth	128 GB/s

2) *Network Configuration*: Network characteristics were configured following the assumptions described in Table 3 of [16].

TABLE II
SIMULATED NETWORK CONFIGURATION

Parameter	Value
Network Bandwidth	100 Mb/s (0.0125 GB/s)
Network Latency	150 ms

3) *Model Generation*: Model characteristics were derived from the LLaMA2-7B architecture, which contains 32 layers, 14 GB of weights, and 128 GB of KV cache [7]. Assuming a uniform distribution across layers, the reference model corresponds to approximately 0.44 GB of weights and 4 GB of KV cache per layer.

To introduce workload diversity, synthetic models were generated by randomizing both the number of layers and layer memory footprints around the reference configuration, as seen in Table III.

TABLE III
SYNTHETIC MODEL GENERATION PARAMETERS

Parameter	Value
Layer Count	Uniform random integer in [32, 62]
Per-layer KV Cache Size	$4 + U(0, 10)$ GB
Per-layer Weight Size	$0.44 + U(0, 1)$ GB

4) *Serving Scenarios*: The evaluation considers two serving scenarios:

- Single-model serving: each inference request requires the execution of a single model. This scenario represents traditional LLM inference workloads.
- Multi-model serving: inference requests may involve multiple models within a single request pipeline. This scenario models modern AI systems such as RAG and agentic workflows.

5) *Workload Distributions*: To evaluate the sensitivity of cache eviction policies to different request patterns, model requests are generated by sampling model identifiers from a model set M of size K . For each request, the simulator selects a model index $i \in \{0, \dots, K - 1\}$ using a pseudo-random number generator initialized with a fixed seed to ensure reproducibility.

Five workload distributions are considered:

- Uniform: Model identifiers are sampled with equal probability, representing workloads with no popularity bias.
- Normal: Model identifiers are sampled from a Gaussian distribution centered at the midpoint of the model zoo with standard deviation $K/6$. This creates moderate locality, where a subset of models receives more requests than others.
- Power Law: Model identifiers are generated using a Zipf distribution with exponent $s = 1.1$. This produces highly skewed popularity patterns in which a small subset of models dominates the request stream, resembling production workloads.
- Poisson: Model identifiers are sampled using a Poisson distribution with parameter $\lambda = K/2$, producing stochastic request patterns with moderate variability.
- Gamma: Model identifiers are sampled from a Gamma distribution with shape parameter $\alpha = 2$ and scale parameter $\beta = 2$, generating burstier request patterns than the Poisson workload.

For the Normal, Poisson, and Gamma distributions, sampled values outside the valid identifier range are clipped to the nearest valid model identifier.

6) *Simulation Parameters*: The simulation is configured using the parameters shown in Table IV. These parameters define the scale of the distributed system, including the number of interacting nodes and the intensity of the workload generated during execution.

TABLE IV
SIMULATION PARAMETERS

Parameter	Value
Number of Clients	10
Number of Workers	4
Requests per Client	20
Number of Models	40

To ensure meaningful cache dynamics, the system is deliberately configured in a memory-constrained regime. Specifically, the aggregated size of all generated models exceeds the total available vRAM capacity of the worker cluster (2560 GB), forcing frequent eviction, transfer, and reloading events during execution. This setup is essential to stress-test cache eviction policies under realistic contention conditions.

C. Evaluation Metrics

We evaluate cache eviction policies using latency-based metrics derived from per-request execution traces. Each metric is computed over all completed inference requests in a given

experiment. All reported results are averaged over 10 independent runs of the same configuration, each using a distinct random seed to account for stochastic variability in workload generation and system scheduling.

- **Median Latency:** The median (50th percentile) inference latency across all requests, providing a robust measure of typical system performance.
- **Tail Latency (P90):** The 90th percentile inference latency, capturing performance under moderate contention.
- **Tail Latency (P99):** The 99th percentile inference latency, capturing worst-case performance under high contention and memory pressure.

D. Plots.

Due to the multi-order-of-magnitude variation between local cache hits and cold-start retrieval delays, the vertical axis of all plots are rendered using a logarithmic base-10 scale to maintain visual granularity across highly competitive configurations.

E. Results

Each cache eviction policy was evaluated under all combinations of workload distribution and serving scenario, resulting in a comprehensive evaluation matrix. To keep the main discussion concise, Research Questions RQ1 V-E1 and RQ2 V-E2 present results for a representative workload distribution, while the complete results for all workload distributions are provided in Appendix A. Research Question RQ3 V-E3 analyzes a set of meaningful workload distributions to quantify the sensitivity of cache eviction policies to different request patterns.

1) *RQ1: Single-Model Serving:* To answer RQ1, we evaluate all cache eviction policies under the Gamma workload distribution. Gamma distributions are commonly used to model bursty request arrivals and temporal locality observed in production AI systems [8]. Since the objective of this work is to minimize inference latency under realistic request patterns, the Gamma workload serves as the representative scenario for comparing policies.

Table V reports the median, P90, and P99 request latencies obtained by each policy. Figure 1 visualizes the histogram of the same results. Results for the remaining workload distributions are provided in the Table IX in Appendix A.

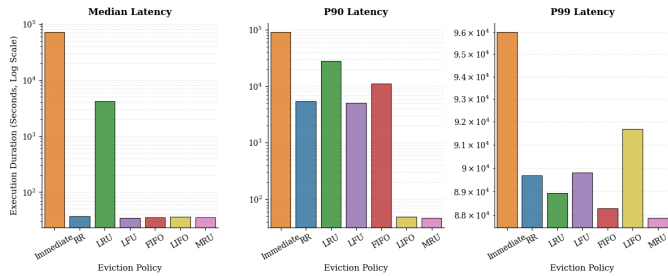


Fig. 1. Comparative evaluation of single-model cache eviction policies across metrics under a Gamma workload distribution.

TABLE V
CACHE EVICTION POLICY PERFORMANCE ACROSS GAMMA
DISTRIBUTION FOR SINGLE MODEL SERVING (SECONDS)

Policy	Metric	Gamma
Immediate	Median	71486.604
	P90	90450.988
	P99	96002.831
Random (RR)	Median	36.706
	P90	5438.482
	P99	89685.935
LRU	Median	4194.429
	P90	28007.873
	P99	88939.287
LFU	Median	34.264
	P90	5100.120
	P99	89809.892
FIFO	Median	35.353
	P90	11139.643
	P99	88290.169
LIFO	Median	36.386
	P90	48.760
	P99	91679.528
MRU	Median	35.228
	P90	46.287
	P99	87881.111

a) *Baseline comparison:* The Immediate policy exhibits substantially higher latency than all caching strategies. Its median latency reaches 71,486.6 seconds, compared to approximately 34–36 seconds for most cache-based policies. This result confirms that retaining model layers across requests is critical for reducing inference latency. Requiring models to be reloaded after every request introduces excessive network transfers and memory movement costs, which dominate end-to-end latency.

b) *Cache policy comparison:* Among the evaluated cache eviction policies, LFU achieves the lowest median latency at 34.3 seconds, making it the best-performing policy in terms of median latency. FIFO, MRU, LIFO, and RR achieve similar median latencies, all within approximately 7% of the LFU result.

In contrast, LRU performs significantly worse, with a median latency of 4,194.4 seconds. This suggests that under the evaluated workload, recency alone is not a reliable predictor of future accesses. The bursty nature of the Gamma workload appears to favor policies that preserve frequently requested models rather than recently requested ones.

Analysis of tail latency reveals a different behavior. While LFU minimizes median latency, MRU and LIFO achieve the lowest P90 latencies, with values of 46.3 and 48.8 seconds, respectively. This indicates that these policies provide more consistent performance across the majority of requests. However, all cache-based policies exhibit similar P99 latencies, ranging between 87,881 and 91,680 seconds. These extreme values suggest that rare cache-miss events requiring model transfers from lower memory tiers dominate worst-case latency regardless of the eviction strategy.

The results indicate that there is no universally optimal cache eviction policy for single-model serving workloads. LRU provides the lowest median latency, while MRU and LIFO achieve the best P90 latency. Since all policies exhibit comparable P99 performance, the choice of policy depends on whether the system prioritizes typical request latency or stronger tail-latency guarantees. This observation is consistent with the no-free-lunch principle that motivates the evaluation of multiple cache eviction strategies.

2) *RQ2: Multi-Model Serving*: To identify the cache eviction policy that minimizes inference latency for multi-model serving workloads, we again use the Gamma workload distribution. As discussed previously, Gamma distributions provide a realistic approximation of production AI traffic due to their ability to capture burstiness and temporal locality in request arrivals [8].

Table VI reports the median, P90, and P99 latency observed for each evaluated policy, while Figure 2 provides a visual comparison across the three metrics. The complete results for all workload distributions are reported in Table X in the Appendix A.

TABLE VI
CACHE EVICTION POLICY PERFORMANCE ACROSS GAMMA
DISTRIBUTION FOR MULTI-MODEL SERVING (SECONDS)

Policy	Metric	Gamma
Immediate	Median	6324.139
	P90	93225.288
	P99	177436.614
Random (RR)	Median	25024.318
	P90	70643.997
	P99	133032.145
LRU	Median	3.573
	P90	75736.099
	P99	157771.372
LFU	Median	0.521
	P90	68049.130
	P99	152688.383
FIFO	Median	0.553
	P90	86703.595
	P99	179287.484
LIFO	Median	3889.786
	P90	69706.348
	P99	182244.693
MRU	Median	0.521
	P90	56759.144
	P99	126341.028

a) *Baseline*: The Immediate policy performs poorly in the multi-model setting, with a median latency of 6324.139 seconds and the second-highest P90 and P99 values among all evaluated policies. Unlike the single-model scenario, however, Immediate is not the worst-performing policy overall. RR and LIFO exhibit worse median latency, while several policies achieve substantially lower latency across all metrics.

b) *Policy Comparison*: Among the evaluated cache eviction policies, LFU and MRU achieve the lowest median latency, both obtaining a value of 0.521 seconds. FIFO follows



Fig. 2. Comparative evaluation of multi-model cache eviction policies across metrics under a Gamma workload distribution.

closely with a median latency of 0.553 seconds, indicating that these three policies are highly effective at preserving the model layers most likely to be reused.

When considering tail latency, MRU achieves the best P90 performance at 56 759.144 seconds, outperforming all other policies. MRU also attains the lowest P99 latency of 126 341.028 seconds, indicating superior performance under adverse cache conditions and heavy contention.

In contrast, RR and LIFO exhibit significantly worse median latency than the other evaluated policies, suggesting that recency- or frequency-aware eviction decisions become increasingly important when requests involve multiple models. Multi-model serving introduces greater competition for memory resources, making informed eviction decisions more valuable than in the single-model setting.

For multi-model serving workloads under constrained memory resources, MRU provides the best overall latency performance. Although LFU and MRU achieve identical median latency, MRU substantially outperforms all other policies in both P90 and P99 latency. Consequently, MRU offers the most favorable balance between typical-case and tail-case performance, making it the most effective cache eviction policy for the evaluated multi-model serving scenario.

3) *RQ3: Workload Distribution Sensitivity*: For this research question, Gamma, Power Law, and Poisson workloads were selected because they represent three common request patterns observed in inference systems: bursty traffic, highly skewed popularity distributions, and stochastic independent arrivals, respectively.

Tables VII and VIII summarize the performance of all cache eviction policies across the three workload distributions for single model and multi-model, respectively.

a) *Single-Model Serving*: In the single-model serving scenario, the workload distribution has a significant impact on policy performance. While LFU, FIFO, MRU, and RR exhibit relatively stable median latencies across all three distributions (approximately 34-38 seconds), LRU shows a substantial degradation under the Gamma workload, where its median latency increases to 4194.429 seconds compared to approximately 37 seconds under Power Law and Poisson workloads.

The largest differences appear in the tail latencies. Under the Gamma workload, MRU and LIFO achieve exceptionally

TABLE VII
CACHE EVICTION POLICY PERFORMANCE MATRIX ACROSS WORKLOAD
DISTRIBUTIONS FOR SINGLE MODEL SERVING (SECONDS)

Policy	Metric	Gamma	Power Law	Poisson
Immediate	Median	71486.604	80520.078	75499.777
	P90	90450.988	93474.005	92209.490
	P99	96002.831	97940.886	97075.498
Random (RR)	Median	36.706	38.057	37.997
	P90	5438.482	69391.012	61588.365
	P99	89685.935	93376.725	93634.384
LRU	Median	4194.429	37.450	37.077
	P90	28007.873	69973.731	59716.996
	P99	88939.287	95418.082	91848.633
LFU	Median	34.264	34.826	36.509
	P90	5100.120	71353.363	60497.859
	P99	89809.892	94099.031	93846.535
FIFO	Median	35.353	37.073	38.279
	P90	11139.643	72406.931	58921.217
	P99	88290.169	94248.201	93038.188
LIFO	Median	36.386	19.825	37.275
	P90	48.760	67486.616	61223.763
	P99	91679.528	92348.055	92712.184
MRU	Median	35.228	38.453	37.014
	P90	46.287	71054.452	59434.516
	P99	87881.111	92678.434	94262.957

low P90 latencies (46.287 and 48.760 seconds, respectively), whereas the same policies experience P90 latencies exceeding 60,000 seconds under the Power Law and Poisson workloads. Similarly, LFU achieves a P90 latency of 5100.120 seconds under Gamma but more than 60,000 seconds under the other distributions.

These results indicate that workload characteristics substantially influence tail latency behaviour, even when median latency remains relatively stable. The Gamma workload appears to favour recency-based policies, while Power Law and Poisson workloads reduce the performance gap between policies.

b) Multi-Model Serving.: The impact of workload distribution is even more pronounced in the multi-model serving scenario. Median latency varies by several orders of magnitude depending on the workload. For example, LFU achieves median latencies of 0.521 seconds under Gamma and 0.553 seconds under Poisson, but increases to 18.866 seconds under the Power Law workload. A similar pattern can be observed for FIFO.

Despite these fluctuations, LFU and FIFO consistently remain among the best-performing policies across all workloads. In contrast, RR and the Immediate baseline exhibit considerably higher median latencies and demonstrate limited adaptability to changes in request patterns. For tail latency, no single policy dominates across all distributions. MRU achieves the lowest P90 and P99 latency under the Gamma workload, FIFO achieves the lowest P90 under the Power Law workload, and LIFO achieves the lowest P90 under the Poisson workload. This suggests that workload characteristics have a stronger influence on tail latency than on median latency.

TABLE VIII
CACHE EVICTION POLICY PERFORMANCE MATRIX ACROSS WORKLOAD
DISTRIBUTIONS FOR MULTI-MODEL SERVING (SECONDS)

Policy	Metric	Gamma	Power Law	Poisson
Immediate	Median	6324.139	17576.689	0.521
	P90	93225.288	97010.697	78292.655
	P99	177436.614	204120.381	206252.066
Random (RR)	Median	25024.318	10165.857	17388.210
	P90	70643.997	70970.390	81664.392
	P99	133032.145	121130.938	148190.850
LRU	Median	3.573	4122.507	7788.317
	P90	75736.099	70430.863	74358.614
	P99	157771.372	139983.605	157497.345
LFU	Median	0.521	18.866	0.553
	P90	68049.130	96510.135	79631.117
	P99	152688.383	254892.300	195370.744
FIFO	Median	0.553	4.708	1.683
	P90	86703.595	59635.289	90677.537
	P99	179287.484	190271.022	194853.334
LIFO	Median	3889.786	6483.125	4178.066
	P90	69706.348	84916.935	63095.040
	P99	182244.693	165221.833	149001.355
MRU	Median	0.521	4305.088	5958.639
	P90	56759.144	81253.545	78314.963
	P99	126341.028	166260.412	172256.418

The experimental results demonstrate that cache eviction policies are sensitive to workload characteristics, particularly with respect to tail latency. Among the evaluated policies, LFU shows the highest degree of robustness, consistently achieving low latency across all workload distributions and both serving scenarios. MRU exhibits similar robustness in the single-model serving scenario, but its performance degrades more noticeably under multi-model serving workloads. Therefore, while workload characteristics significantly influence cache performance, LFU emerges as the most resilient policy across the full range of evaluated conditions.

VI. CONCLUSION

This work studied the impact of classical cache eviction policies in the context of distributed inference systems for modern AI workloads. We focused on a pseudo-stable serving environment where large models are distributed across a hierarchical memory system and inference requests trigger frequent data movement between disk, RAM, and vRAM. In this setting, cache management decisions directly influence end-to-end inference latency.

To analyze this problem, we developed an event-driven simulator in Go that models clients, a centralized coordinator, distributed workers, and a hierarchical memory system with configurable eviction policies. The simulator enabled controlled evaluation of multiple cache strategies under a variety of workload distributions and both single-model and multi-model serving scenarios.

Our experimental results show that no single cache eviction policy is universally optimal across all workloads and serving configurations. In single-model serving, LFU consistently

achieves the lowest median latency, while MRU and LIFO provide better performance in tail latency under specific workload conditions. In multi-model serving, memory contention significantly amplifies performance differences, where LFU and MRU emerge as the most effective policies, with MRU achieving the best balance between median and tail latency under the evaluated Gamma workload.

Across all experiments, workload distribution is a dominant factor shaping cache performance, particularly for tail latency. Bursty and highly skewed workloads amplify differences between policies, while more uniform or stochastic distributions reduce performance gaps. This demonstrates that cache behavior in distributed inference systems cannot be understood independently of workload characteristics, as both median and tail latency are strongly workload-dependent.

Overall, this work highlights that classical cache eviction policies, while effective in traditional systems, exhibit fundamentally different behavior in distributed inference environments due to hierarchical memory constraints, large model sizes, and multi-model execution patterns. Consequently, LFU and MRU emerge as strong general-purpose candidates, but their effectiveness is highly dependent on the serving scenario and workload characteristics.

A. Limitations

Despite the insights provided by this work, several limitations should be acknowledged. First, the simulator relies on simplified cost models for network, disk, and memory transfers, which do not fully capture the complexity of real-world distributed GPU clusters. Second, workload generation is based on synthetic statistical distributions, which may not fully reflect production traces observed in modern inference serving systems. Recent studies such as ServerGen [8] suggest that real workloads exhibit more nuanced and multi-dimensional patterns than those captured in standard distributions.

Additionally, the current model assumes deterministic execution costs and does not incorporate dynamic system effects such as congestion, hardware variability, or adaptive batching strategies commonly used in production inference systems. Finally, cache policies are evaluated in isolation, without considering potential interactions with scheduling [4], batching [10], or request routing policies [2].

B. Future Work

Future work should focus on integrating real-world workload traces, such as those characterized in ServerGen [8], to improve the fidelity of the evaluation environment. Extending the simulator to incorporate adaptive or learned cache eviction policies would also be a promising direction, particularly policies that dynamically adjust based on observed workload characteristics.

Another important direction is the joint optimization of cache management with scheduling and batching strategies, as these components are tightly coupled in real inference serving systems. Finally, exploring reinforcement learning-based or workload-aware hybrid policies could help bridge the

gap between classical heuristics and modern adaptive systems, potentially leading to more robust performance across diverse deployment scenarios.

REFERENCES

- [1] Zhengchao Wang et al. *RAGPulse: An Open-Source RAG Workload Trace to Optimize RAG Serving Systems*. 2025. arXiv: 2511.12979 [cs.LG]. URL: <https://arxiv.org/abs/2511.12979>.
- [2] Srikanta Prasad S V and Utkarsh Arora. *Scalable Inference Architectures for Compound AI Systems: A Production Deployment Study*. 2026. arXiv: 2604.25724 [cs.AI]. URL: <https://arxiv.org/abs/2604.25724>.
- [3] Konstantinos Papaioannou and Thaleia Dimitra Doudali. *Systems for LLMs Are Old News: Multimodality Is Redefining Everything We Know*. URL: <https://2025.eurosys.org/posters/eurosys25posters-paper96.pdf>.
- [4] Bingyang Wu et al. *Fast Distributed Inference Serving for Large Language Models*. 2024. arXiv: 2305.05920 [cs.LG]. URL: <https://arxiv.org/abs/2305.05920>.
- [5] Ying Sheng et al. *FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU*. 2023. arXiv: 2303.06865 [cs.LG]. URL: <https://arxiv.org/abs/2303.06865>.
- [6] Lingfan Yu, Jinkun Lin, and Jinyang Li. “Stateful Large Language Model Serving with Pensieve”. In: *Proceedings of the Twentieth European Conference on Computer Systems*. EuroSys ’25. ACM, Mar. 2025, pp. 144–158. DOI: 10.1145/3689031.3696086. URL: <http://dx.doi.org/10.1145/3689031.3696086>.
- [7] Qiaozhi He and Zhihua Wu. *Efficient LLM Inference with Kcache*. 2024. arXiv: 2404.18057 [cs.CL]. URL: <https://arxiv.org/abs/2404.18057>.
- [8] Yuxing Xiang et al. *ServeGen: Workload Characterization and Generation of Large Language Model Serving in Production*. 2025. arXiv: 2505.09999 [cs.DC]. URL: <https://arxiv.org/abs/2505.09999>.
- [9] Weiping Yu et al. *RAGDoll: Efficient Offloading-based Online RAG System on a Single GPU*. 2025. arXiv: 2504.15302 [cs.DC]. URL: <https://arxiv.org/abs/2504.15302>.
- [10] Sohaib Ahmad et al. “Proteus: A High-Throughput Inference-Serving System with Accuracy Scaling”. In: *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. ASPLOS ’24. La Jolla, CA, USA: Association for Computing Machinery, 2024, pp. 318–334. ISBN: 9798400703720. DOI: 10.1145/3617232.3624849. URL: <https://doi.org/10.1145/3617232.3624849>.
- [11] Hussein Al-Zoubi, Aleksandar Milenkovic, and Milena Milenkovic. “Performance evaluation of cache replacement policies for the SPEC CPU2000 benchmark suite”. In: *Proceedings of the 42nd Annual ACM Southeast Conference*. ACMSE ’04. Huntsville, Alabama: Asso-

ciation for Computing Machinery, 2004, pp. 267–272. ISBN: 1581138709. DOI: 10.1145/986537.986601. URL: <https://doi.org/10.1145/986537.986601>.

- [12] Sem Borst, Varun Gupta, and Anwar Walid. “Distributed Caching Algorithms for Content Distribution Networks”. In: *2010 Proceedings IEEE INFOCOM*. 2010, pp. 1–9. DOI: 10.1109/INFCOM.2010.5461964.
- [13] *Authors’ implementation of Simulator in Go*. URL: <https://github.com/AndreaQuirozO/DistributedCachedForPseudStableSystems>.
- [14] NVIDIA. *Introduction to NVIDIA DGX H100/H200 Systems*. URL: <https://docs.nvidia.com/dgx/dgxm100-user-guide/introduction-to-dgxm100.html>.
- [15] NVIDIA. *NVIDIA H100 GPU*. URL: <https://www.nvidia.com/en-us/data-center/h100/>.
- [16] Vincent Gramoli et al. “Diablo: A Benchmark Suite for Blockchains”. In: *Proceedings of the Eighteenth European Conference on Computer Systems*. EuroSys ’23. Rome, Italy: Association for Computing Machinery, 2023, pp. 540–556. ISBN: 9781450394871. DOI: 10.1145/3552326.3567482. URL: <https://doi.org/10.1145/3552326.3567482>.

APPENDIX

TABLE IX
SINGLE MODEL CACHE EVICTION POLICY PERFORMANCE MATRIX ACROSS WORKLOAD DISTRIBUTIONS (SECONDS)

Policy	Metric	Uniform	Normal	Power Law	Poisson	Gamma
Immediate	Median	60845.910	72205.890	80520.078	75499.777	71486.604
	P90	91338.811	92513.000	93474.005	92209.490	90450.988
	P99	100958.629	97898.673	97940.886	97075.498	96002.831
Random (RR)	Median	40.773	41.163	38.057	37.997	36.706
	P90	74167.467	69983.003	69391.012	61588.365	5438.482
	P99	92700.491	94403.281	93376.725	93634.384	89685.935
LRU	Median	42.617	38.788	37.450	37.077	4194.429
	P90	75408.135	68383.160	69973.731	59716.996	28007.873
	P99	94482.340	94235.204	95418.082	91848.633	88939.287
LFU	Median	2898.156	36.822	34.826	36.509	34.264
	P90	67640.827	70432.683	71353.363	60497.859	5100.120
	P99	83279.631	92600.461	94099.031	93846.535	89809.892
FIFO	Median	42.575	39.907	37.073	38.279	35.353
	P90	74135.114	70761.816	72406.931	58921.217	11139.643
	P99	95791.796	93697.521	94248.201	93038.188	88290.169
LIFO	Median	34.518	16.108	19.825	37.275	36.386
	P90	65085.287	62489.858	67486.616	61223.763	48.760
	P99	85792.984	82568.575	92348.055	92712.184	91679.528
MRU	Median	41.561	32.159	38.453	37.014	35.228
	P90	74960.960	69415.225	71054.452	59434.516	46.287
	P99	95908.237	94213.418	92678.434	94262.957	87881.111

TABLE X
MULTI-MODEL CACHE EVICTION POLICY PERFORMANCE MATRIX ACROSS WORKLOAD DISTRIBUTIONS (SECONDS)

Policy	Metric	Uniform	Normal	Power Law	Poisson	Gamma
Immediate	Median	88936.407	0.537	17576.689	0.521	6324.139
	P90	246182.215	90783.603	97010.697	78292.655	93225.288
	P99	322399.658	213102.807	204120.381	206252.066	177436.614
Random (RR)	Median	21979.918	13175.687	10165.857	17388.210	25024.318
	P90	113733.772	97719.041	70970.390	81664.392	70643.997
	P99	189533.345	163893.641	121130.938	148190.850	133032.145
LRU	Median	4319.312	10987.859	4122.507	7788.317	3.573
	P90	60087.242	67529.442	70430.863	74358.614	75736.099
	P99	134258.769	166620.204	139983.605	157497.345	157771.372
LFU	Median	10346.627	3.476	18.866	0.553	0.521
	P90	75685.963	93288.360	96510.135	79631.117	68049.130
	P99	180621.083	269875.631	254892.300	195370.744	152688.383
FIFO	Median	0.490	0.482	4.708	1.683	0.553
	P90	40606.987	43879.518	59635.289	90677.537	86703.595
	P99	127315.861	119090.593	190271.022	194853.334	179287.484
LIFO	Median	2864.827	3325.603	6483.125	4178.066	3889.786
	P90	70595.509	79965.883	84916.935	63095.040	69706.348
	P99	203883.408	162862.777	165221.833	149001.355	182244.693
MRU	Median	3.658	10879.649	4305.088	5958.639	0.521
	P90	85679.758	86821.748	81253.545	78314.963	56759.144
	P99	163613.870	185667.292	166260.412	172256.418	126341.028